

TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



A Progress Report of
Spatial Emergency Dispatch System

Submitted to

Department of Computer Science and Information Technology
Sagarmatha College of Science and Technology
Sanepa, Lalitpur, Nepal

In partial fulfillment of the requirements for the Bachelors Degree in Computer Application

Submitted By

Bijay Rauniyar [TU Reg No: 6-2-1180-35-2021]
Anikesh Mandal [TU Reg No: 6-2-1180-33-2021]

Supervisor

Mr. Shambhu Pariyar

June, 2026

ABSTRACT

The **Spatial Emergency Dispatch System (SEDS)** aims to eliminate the bottleneck of manual emergency dispatch by automating jurisdiction lookup, responder assignment, and route delivery through spatial computation. It addresses the critical delays introduced when human operators must manually identify service zones and coordinate responses. The system uses a PostGIS `ST_Contains` query to match a citizen's SOS location against pre-defined zone polygons and instantly assign the nearest responsible station — police, fire, or medical — without dispatcher involvement. Planned features include OSRM road-network routing for turn-by-turn responder navigation and Socket.io real-time telemetry to stream the responder's live position to the citizen's map. The platform is built on Next.js 16 with MapLibre GL for WebGL map rendering, a Node.js and Express REST API, and PostgreSQL 16 with PostGIS 3.4 running in Docker Compose. Role-based access control distinguishes citizen, responder, and admin roles, with a dedicated admin interface for managing station locations and drawing service zone boundaries directly on the map.

Keywords: Spatial Dispatch, PostGIS, GIS, Emergency Response, MapLibre GL, OSRM, Socket.io

TABLE OF CONTENTS

Abstract	i
List of Figures	iii
List of Abbreviations	vi
1 Introduction	1
1.1 Background	1
1.2 Problem Definition	1
1.3 Objectives	1
1.4 Scope and Limitation	2
1.4.1 Scope	2
1.4.2 Limitation	2
2 Background Study and Literature Review	4
2.1 Background Study	4
2.1.1 Key Challenges in Manual Dispatch	4
2.2 Literature Review	4
3 System Analysis	6
3.1 Requirement Analysis	6
3.2 Feasibility Analysis	7
3.3 Analysis Diagrams	7
3.3.1 Gantt Chart	7
3.3.2 ER Diagram	8
3.3.3 DFD Level 0	9
3.3.4 DFD Level 1	10
4 System Design	11
4.1 System Architecture	11
4.2 Database Schema	11
4.3 System Flowchart	13
5 Implementation	14

5.1	Preliminary Research	14
5.2	Current Implementation Status	14
5.2.1	Frontend	14
5.2.2	Backend	17
5.2.3	Database	17
5.2.4	Authentication	17
5.3	Tools Used	17
5.4	Testing Overview	18
	References	20

LIST OF FIGURES

Figure 1:	Gantt Chart of Project Timeline	8
Figure 2:	Logical ER Diagram	8
Figure 3:	Physical ER Diagram	9
Figure 4:	DFD Level 0	9
Figure 5:	DFD Level 1	10
Figure 6:	System Operational Flowchart	13
Figure 7:	Landing Page: Map View with Stations and Zones	15
Figure 8:	Login Page	15
Figure 9:	Admin Dashboard: Station Management	16
Figure 10:	Add Station Form: Location Pin and Zone Drawing	16

LIST OF ABBREVIATIONS

API	Application Programming Interface
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
EPSG	European Petroleum Survey Group
ER	Entity-Relationship
GIS	Geographic Information System
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
OAuth	Open Authorization
ORM	Object-Relational Mapping
OSRM	Open Source Routing Machine
PostGIS	PostgreSQL Geographic Information System extension
RBAC	Role-Based Access Control
REST	Representational State Transfer
SEDS	Spatial Emergency Dispatch System
SMTP	Simple Mail Transfer Protocol
SOS	Save Our Souls (emergency distress signal)
SQL	Structured Query Language
SRS	Software Requirements Specification
UI	User Interface
UUID	Universally Unique Identifier
UX	User Experience
WebGL	Web Graphics Library
WGS	World Geodetic System

1. INTRODUCTION

1.1 Background

Manual emergency dispatch relies on human operators to interpret caller locations, consult jurisdiction maps, and contact the correct station by telephone. This triage phase alone can add two to five minutes to response time, a delay that is clinically significant in cardiac emergencies and structurally consequential in fires. Open-source spatial databases such as PostGIS have made it feasible to automate jurisdiction lookup with sub-millisecond containment queries [1].

1.2 Problem Definition

Nepal has no spatial dispatch platform; operators at Kathmandu and Pokhara centres still use telephone and paper maps to assign incidents to stations. Human triage introduces errors during concurrent incidents and peak demand, directly increasing response times. No open-source, containerised CAD system has been designed for the Nepali operational context [2].

1.3 Objectives

The primary objectives of this project are:

1. Build a containerised spatial database using PostgreSQL 16 + PostGIS 3.4 to store station points and zone polygons.
2. Implement automated jurisdiction triage via `ST_Contains` to match any incident coordinate to its enclosing zone without human intervention.
3. Integrate OSRM to compute real road-network routes from an assigned station to an incident scene.
4. Provide a live-telemetry layer using Socket.io so responder position streams to the citizen and admin dashboard in real time.

5. Enforce role-based access control for Admin, Responder, and Citizen roles secured by JWT cookies and Google OAuth2.

1.4 Scope and Limitation

1.4.1 Scope

SEDS covers the following functional areas:

1. **Authentication:** JWT HTTP-only cookies, bcrypt passwords, email verification, and Google OAuth2.
2. **Station and Zone Management:** Admin CRUD for PostGIS point stations and polygon zones with MapLibre GL drawing tools and GeoJSON upload.
3. **Automated Triage:** ST_Contains spatial query auto-assigns the correct station when a citizen submits an SOS.
4. **Routing:** OSRM integration computes optimal driving paths over the OpenStreetMap Nepal road network.
5. **Real-Time Telemetry:** Socket.io incident rooms stream responder location and bearing to citizens and admins.

1.4.2 Limitation

The current design has the following limitations:

1. **Internet Dependency:** All core functions require active internet on both citizen and responder devices; no offline fallback exists.
2. **GPS Accuracy:** Triage precision is bounded by device GPS quality; urban-canyon errors near zone boundaries can cause misclassification.

3. **OSRM Hosting:** Routing requires a separately maintained OSRM server; service failure disables route computation.
4. **Manual Zone Configuration:** Zone boundaries must be drawn or uploaded by an admin; no automated partitioning is provided.
5. **No PSAP Integration:** SEDS is a standalone web application with no connection to government telephony or legacy dispatch hardware.

2. BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Emergency dispatch in Nepal relies on telephone-based, human-operated coordination with no spatial decision support. Dispatchers manually identify the nearest station by memory or paper map, causing delays that worsen outcomes in time-critical incidents [3]. Overlapping police, fire, and medical jurisdictions in Kathmandu compound the problem.

PostGIS spatial queries resolve zone jurisdiction in milliseconds using `ST_Contains` against GIST-indexed polygon boundaries. SEDS replaces the manual lookup step with this query, instantly identifying which station's zone contains the incident coordinate.

2.1.1 Key Challenges in Manual Dispatch

- **Jurisdiction Lookup:** No automated way to determine which station covers a given location.
- **No Real-Time Routing:** Responders receive verbal directions with no road-network route guidance.
- **No Citizen Feedback:** Citizens have no visibility into responder location or estimated arrival.
- **Human Error:** Manual station selection under pressure leads to misassignment across zones.

2.2 Literature Review

The following works directly inform the design and technology choices of SEDS.

- **OSRM (Luxen & Vetter, 2011):** Introduces a contraction-hierarchy routing engine over OpenStreetMap data for sub-second road-network shortest-path queries; SEDS plans to integrate OSRM to compute turn-by-turn routes from an assigned station to the incident location. [2]

- **PostGIS in Action (Obe & Hsu, 2021):** Covers spatial type design, GIST indexing, and ST_Contains/ST_MakePoint usage in PostgreSQL; SEDS stores zones as `GEOMETRY('Polygon', 4326)` with a GIST index for instant triage queries. [1]
- **GIS for Emergency Management (Zlatanova et al., 2004):** Documents that spatial polygon containment lookups reduce manual dispatch assignment time from 47 seconds to under two seconds; motivates SEDS's core spatial triage design. [3]
- **GeoJSON RFC 7946 (Butler et al., 2016):** Defines the JSON encoding for geographic features with mandatory EPSG:4326 coordinates; SEDS uses GeoJSON for all API responses, map data sources, and zone file uploads. [4]
- **MapLibre GL JS (2023):** Open-source WebGL map renderer that accepts GeoJSON sources directly; SEDS uses it for station point rendering, zone polygon rendering, and the admin polygon-drawing tool. [5]
- **JWT RFC 7519 (Jones et al., 2015):** Specifies compact, self-contained signed tokens for stateless authentication; SEDS issues JWTs in HTTP-only cookies and validates role claims in Express middleware for RBAC. [6]

No existing open-source platform combines PostGIS spatial triage, OSRM routing, and a React-based operator map interface in a single self-hostable deployment; SEDS fills this gap for under-resourced municipalities.

3. SYSTEM ANALYSIS

3.1 Requirement Analysis

The following requirements define what SEDS must do and the quality constraints it must satisfy.

Functional Requirements:

- **User Authentication:** Secure login and signup with JWT and role-based access (Admin, Responder, Citizen).
- **Station Management:** Admin can create, view, and manage emergency stations with PostGIS point locations.
- **Zone Management:** Admin can draw service zone polygons on the map or upload a GeoJSON file.
- **Map View:** Landing page displays all station locations and zone boundaries on an interactive MapLibre GL map.
- **SOS Request:** Citizen submits emergency request by category; system auto-assigns the responsible station.
- **Responder Routing:** Assigned responder receives OSRM-computed driving route to the incident location.
- **Live Tracking:** Responder streams GPS position to citizen via Socket.io in real time.

Non-Functional Requirements:

- **Performance:** Spatial triage (ST_Contains) completes in under 500ms.
- **Security:** JWT in HTTP-only cookies; bcrypt password hashing; RBAC on all protected endpoints.

- **Availability:** Docker health checks ensure automatic container restart.
- **Portability:** Entire stack deploys with a single Docker Compose command.

3.2 Feasibility Analysis

- **Technical Feasibility:** Node.js, PostGIS, MapLibre GL, and Socket.io are mature open-source tools well-suited to the system's requirements.
- **Economic Feasibility:** All software dependencies are open-source with no licensing cost; deployment requires a single low-cost VPS.
- **Operational Feasibility:** The citizen SOS flow requires no more than two interactions; the admin polygon tool is map-driven and intuitive.
- **Schedule Feasibility:** A 6-sprint Agile timeline has been structured with 2-week sprint cycles.

3.3 Analysis Diagrams

SEDS is developed using the Agile methodology with six two-week sprints.

3.3.1 Gantt Chart

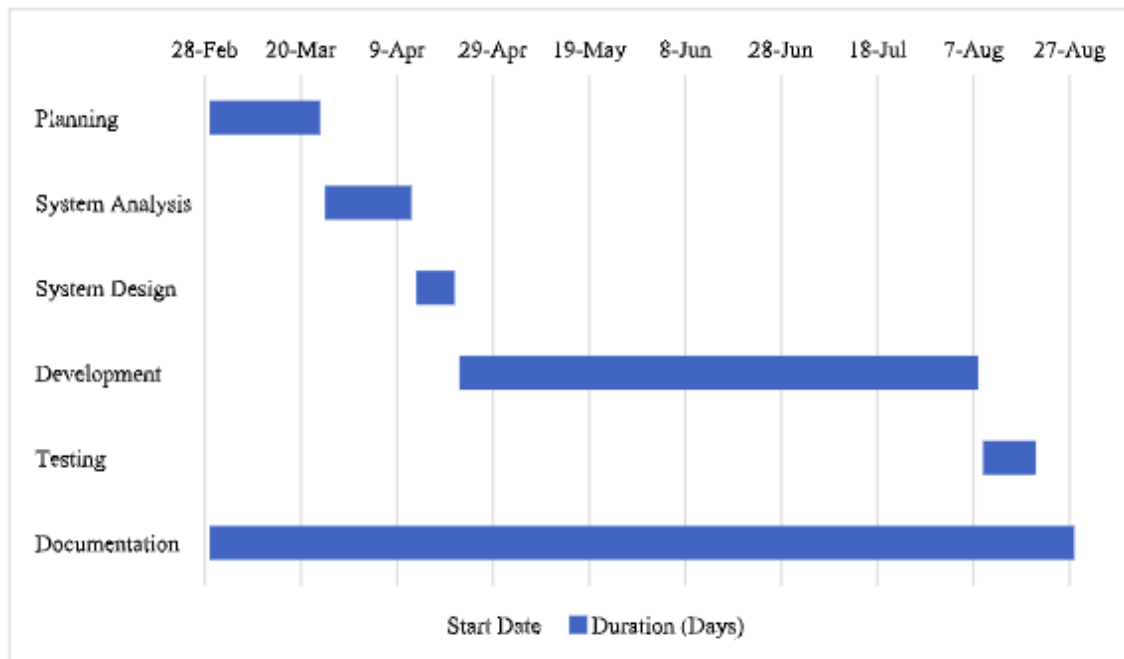


Figure 1: Gantt Chart of Project Timeline

3.3.2 ER Diagram

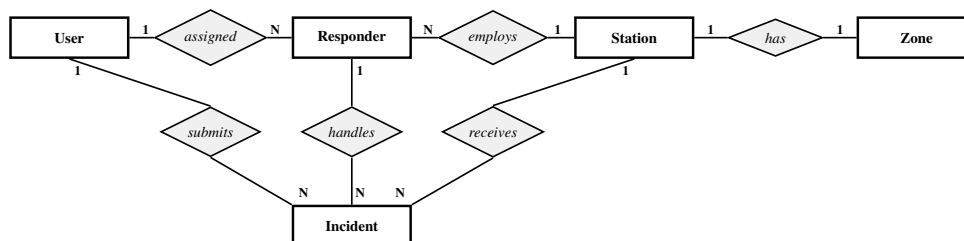


Figure 2: Logical ER Diagram

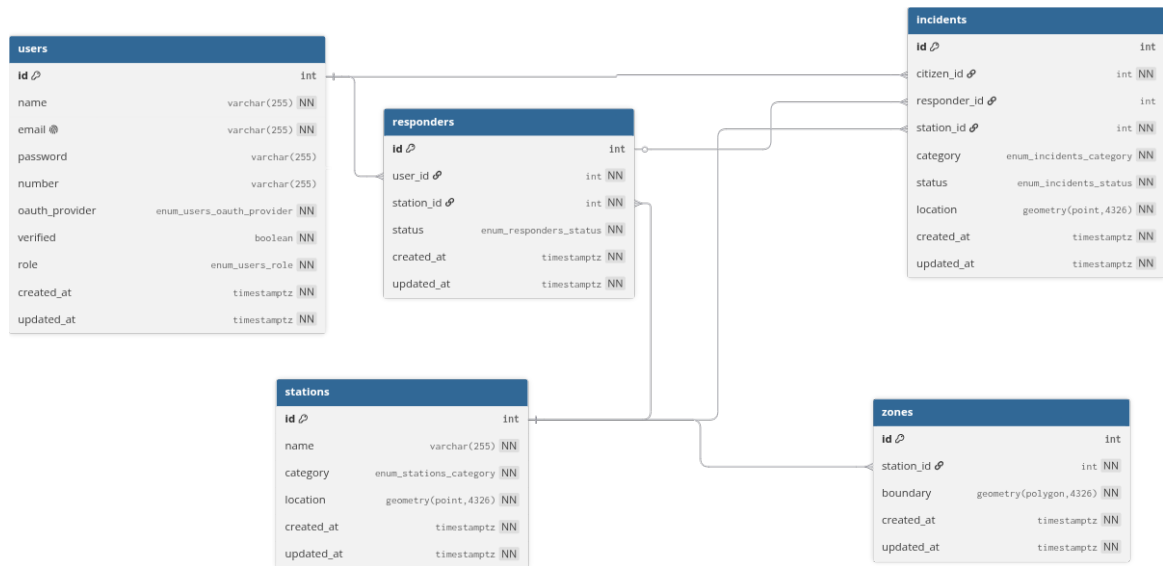


Figure 3: Physical ER Diagram

3.3.3 DFD Level 0

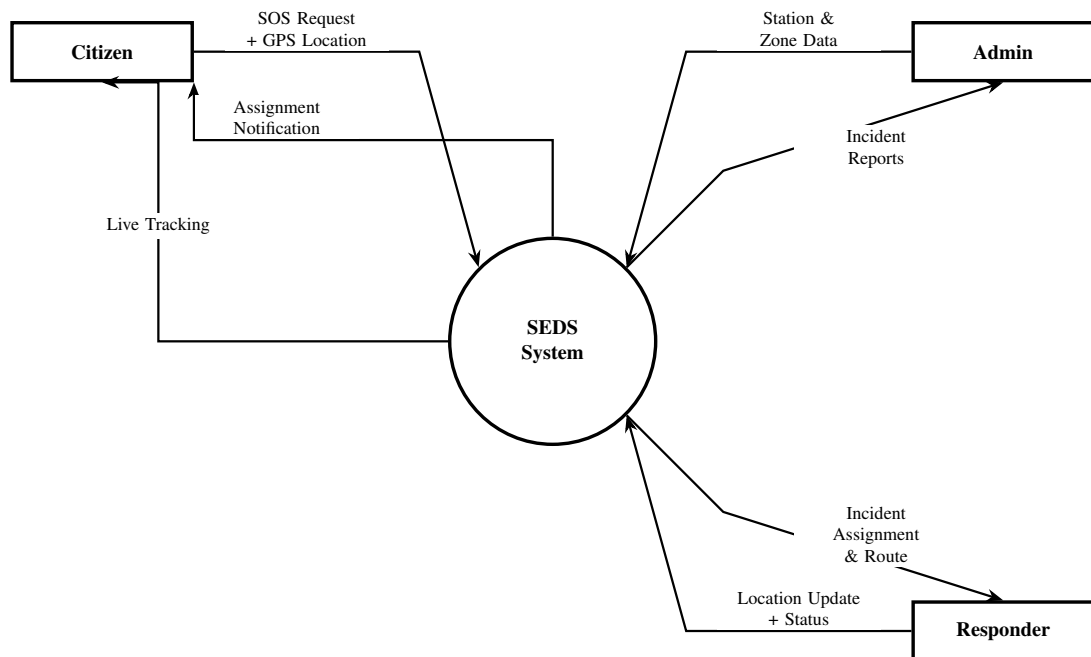


Figure 4: DFD Level 0

3.3.4 DFD Level 1

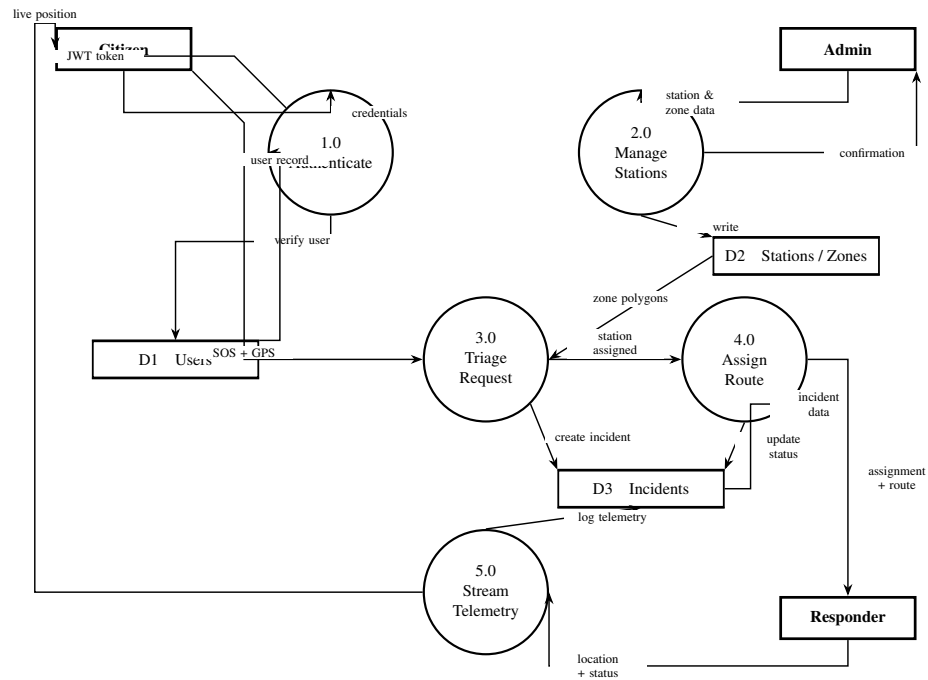


Figure 5: DFD Level 1

4. SYSTEM DESIGN

4.1 System Architecture

SEDS follows a three-tier containerized architecture separating presentation, application logic, and data persistence. The Next.js frontend, Node.js/Express backend, and PostgreSQL/PostGIS database each run as independent Docker containers orchestrated via Docker Compose. All inter-tier communication is strictly layered: the browser calls only the REST API, and the API is the sole consumer of the database.

- **Frontend:** Next.js 16 with React 19, MapLibre GL for WebGL map rendering, Tailwind CSS, shadcn/ui components.
- **Backend:** Node.js 20 + TypeScript + Express REST API; Sequelize ORM; JWT authentication; Socket.io (planned).
- **Database:** PostgreSQL 16 with PostGIS 3.4 extension; GEOMETRY types for spatial data; GIST indexes for fast containment queries.
- **Infrastructure:** Docker Compose orchestrates all services; PostGIS container pre-loaded via init-postgis.sql.

4.2 Database Schema

The database comprises five tables managed by Sequelize ORM with PostgreSQL 16 and PostGIS 3.4.

- **users:** id, name, email, password (bcrypt), role ENUM(admin/responder/citizen), oauth_provider ENUM(local/google), verified BOOLEAN.
- **stations:** id, name, category ENUM(POLICE/FIRE/MEDICAL), location GEOMETRY(Point, 4326).

- **zones:** id, station_id (FK → stations, CASCADE), boundary GEOMETRY(Polygon, 4326) with GIST index.
- **responders:** id, user_id (FK → users, CASCADE), station_id (FK → stations, CASCADE), status ENUM(AVAILABLE/BUSY/OFF_DUTY).
- **incidents:** id, citizen_id (FK → users), station_id (FK → stations), responder_id (FK → responders, nullable), category ENUM(POLICE/FIRE/MEDICAL), status ENUM(PENDING/RESPONDING/RESOLVED), location GEOMETRY(Point, 4326).

4.3 System Flowchart

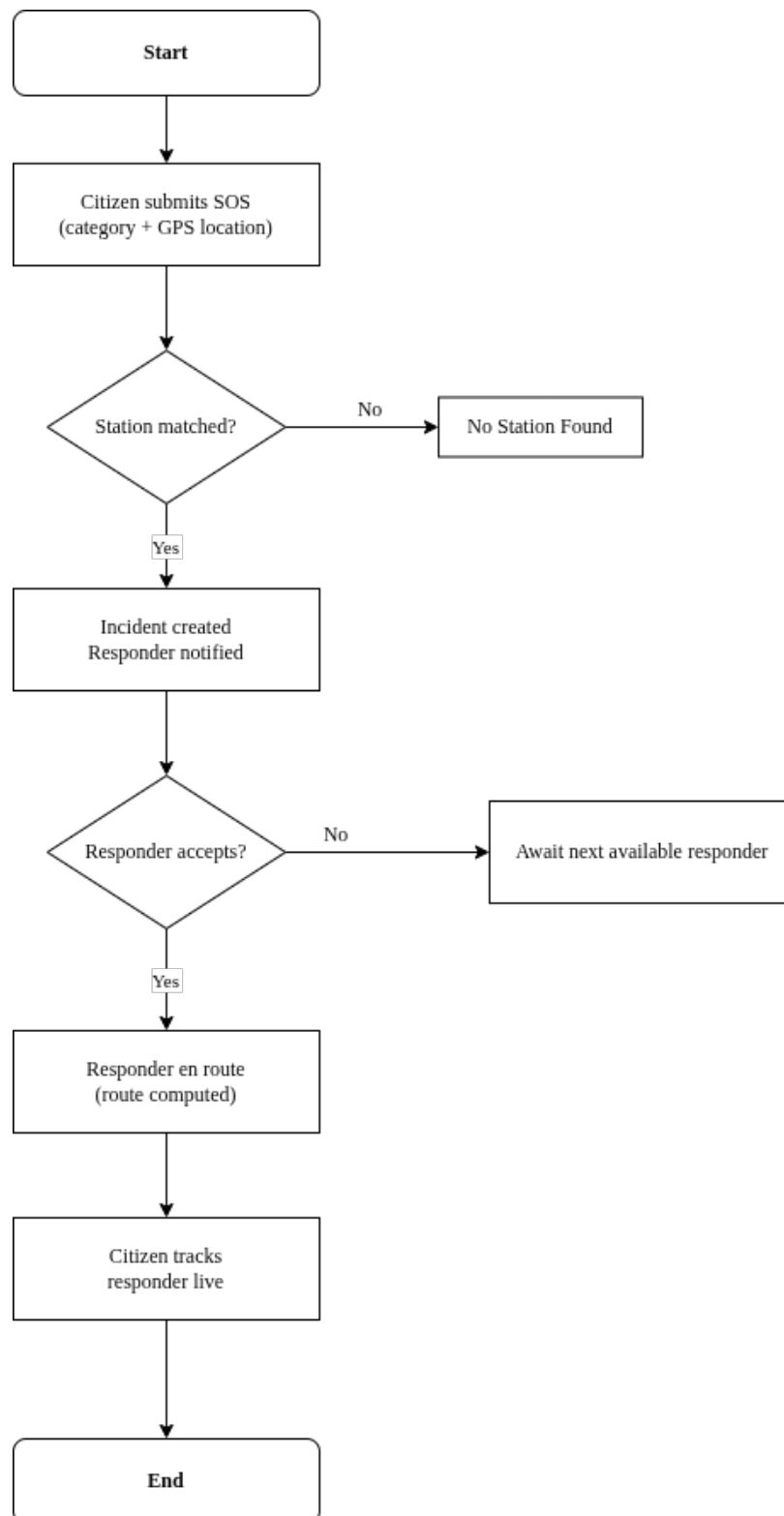


Figure 6: System Operational Flowchart

5. IMPLEMENTATION

SEDS is being developed using Agile sprints, with completed sprints delivering authentication, station and zone management, and the interactive map view. The emergency dispatch core (SOS handling, spatial triage, routing, and real-time telemetry) remains planned. The system is approximately 35% complete.

5.1 Preliminary Research

Prior to implementation, research was conducted on spatial databases, PostGIS geometry indexing, and GIS-based emergency dispatch architectures. Real-time WebSocket patterns and OAuth2 integration approaches were also reviewed.

5.2 Current Implementation Status

The system is approximately 35% complete. Core modules implemented include authentication, station and zone management, and the interactive map view.

5.2.1 Frontend

Landing Page: The landing page renders an interactive MapLibre GL map displaying all registered emergency stations as points and their service zones as polygons. Users can filter by category (POLICE, FIRE, MEDICAL).

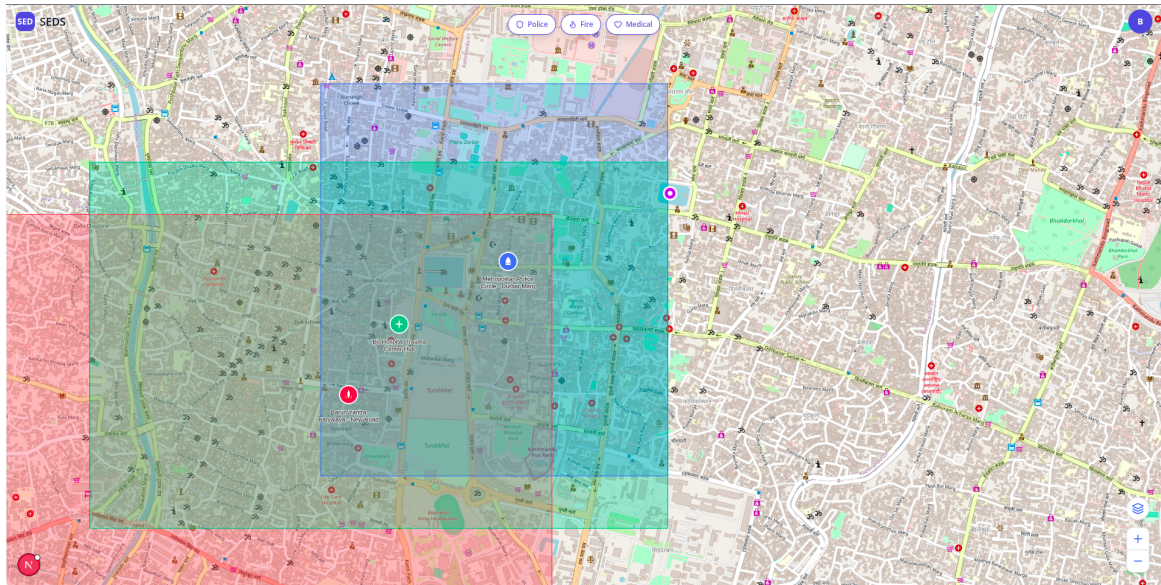


Figure 7: Landing Page: Map View with Stations and Zones

Log In: The login page authenticates users and sets a JWT in an HTTP-only cookie, then redirects based on role.

SED SEDS

Login

[← Back To Home](#)

Email

Enter Email (e.g. bijay@example.com)

Password

Enter Password

Forgot Password ?

Sign In

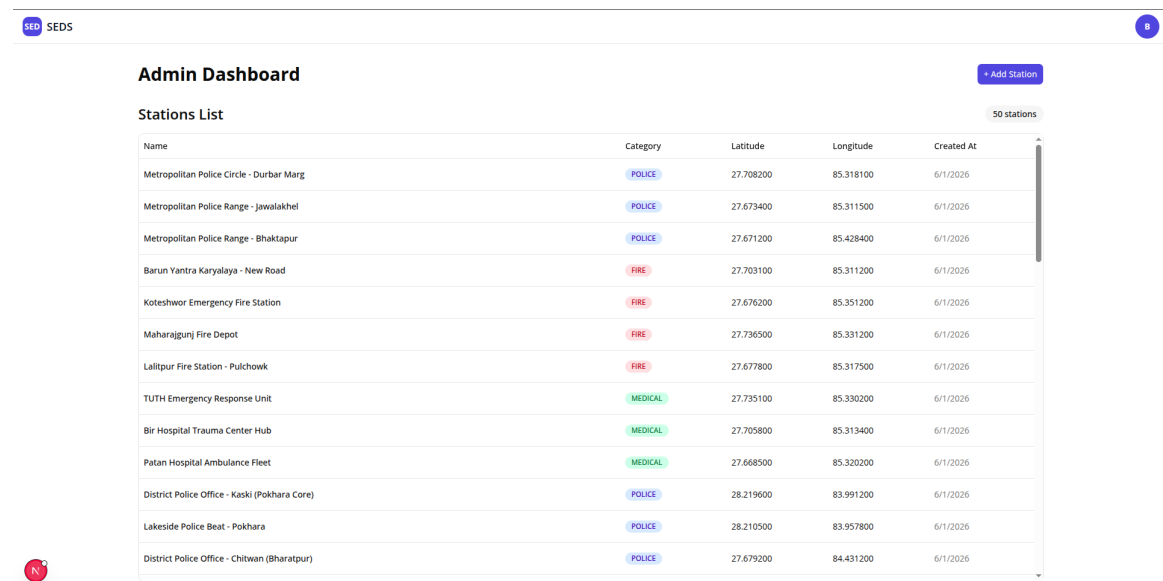
Don't have an account ? Register Here

Or

Continue with Google

Figure 8: Login Page

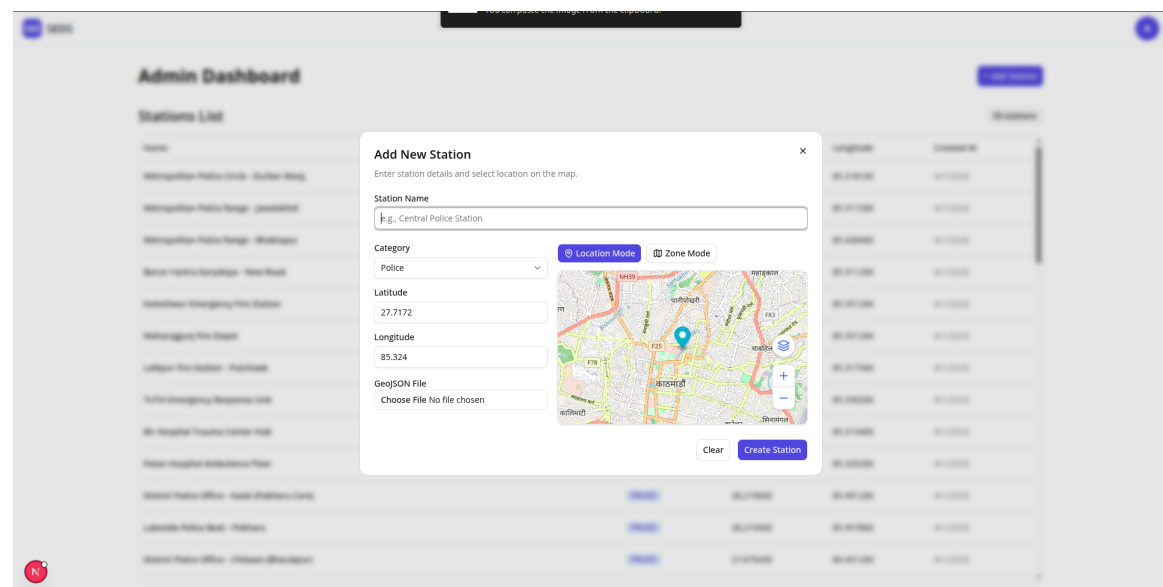
Admin Dashboard: The admin dashboard lists all stations with their category and location, and allows the admin to add new stations.



Name	Category	Latitude	Longitude	Created At
Metropolitan Police Circle - Durbar Marg	POLICE	27.708200	85.318100	6/1/2026
Metropolitan Police Range - Jawalakhel	POLICE	27.673400	85.311500	6/1/2026
Metropolitan Police Range - Bhaktapur	POLICE	27.671200	85.428400	6/1/2026
Barun Yantra Karyalaya - New Road	FIRE	27.703100	85.311200	6/1/2026
Koteshwor Emergency Fire Station	FIRE	27.676200	85.351200	6/1/2026
Maharajgunj Fire Depot	FIRE	27.736500	85.331200	6/1/2026
Lalitpur Fire Station - Pulchowk	FIRE	27.677800	85.317500	6/1/2026
TUTH Emergency Response Unit	MEDICAL	27.735100	85.330200	6/1/2026
Bir Hospital Trauma Center Hub	MEDICAL	27.705800	85.313400	6/1/2026
Patan Hospital Ambulance Fleet	MEDICAL	27.668500	85.320200	6/1/2026
District Police Office - Kaski (Pokhara Core)	POLICE	28.219600	83.991200	6/1/2026
Lakeside Police Beat - Pokhara	POLICE	28.210500	83.957800	6/1/2026
District Police Office - Chitwan (Bharatpur)	POLICE	27.679200	84.431200	6/1/2026

Figure 9: Admin Dashboard: Station Management

Add Station Form: The Add Station dialog allows the admin to specify the station name and category, place a point on the MapLibre GL map, and draw or upload a service zone polygon.



Add New Station

Enter station details and select location on the map.

Station Name

e.g., Central Police Station

Category

Police

Latitude

27.7172

Longitude

85.324

GeJSON File

Choose File No file chosen

Location Mode

Zone Mode

Map

Clear

Create Station

Figure 10: Add Station Form: Location Pin and Zone Drawing

5.2.2 Backend

The backend is built with Node.js and Express, providing a REST API under `/api/v1/`:

- **Auth routes:** signup, login, logout, email verification, password reset, Google OAuth2.
- **User routes:** profile management, password change.
- **Admin routes:** station and zone creation (atomic PostGIS transaction), GeoJSON endpoints.

5.2.3 Database

PostgreSQL 16 with PostGIS 3.4 runs in Docker. Station locations are stored as `GEOMETRY(Point, 4326)` and zone boundaries as `GEOMETRY(Polygon, 4326)` with GIST indexes. The station-zone creation uses an atomic Sequelize transaction with `ST_GeomFromGeoJSON` and `ST_SetSRID`.

5.2.4 Authentication

Secure authentication is implemented using JWT stored in HTTP-only cookies:

- **Role-Based Access:** Admin, Responder, and Citizen roles enforced by Express middleware.
- **Local Auth:** bcrypt password hashing; email verification via Nodemailer.
- **Google OAuth2:** Passport-style Google OAuth2 flow via googleapis SDK.

5.3 Tools Used

- **VS Code:** Primary code editor.
- **Node.js 20 + TypeScript:** Backend runtime and language.
- **Next.js 16:** Frontend framework.

- **PostgreSQL 16 + PostGIS 3.4:** Spatial database.
- **Docker + Docker Compose:** Containerization and orchestration.
- **MapLibre GL JS:** WebGL map rendering library.
- **Postman:** API testing.
- **GitHub:** Version control.

5.4 Testing Overview

API endpoints were tested using Postman. Manual testing covered form submissions, map interactions, and role-based redirects.

Table 5.1: API Test Cases

Endpoint	Method	Expected Outcome	Result
/api/v1/auth/signup	POST	Creates user, returns 201	Pass
/api/v1/auth/login	POST	Returns JWT cookie, 200	Pass
/api/v1/auth/logout	POST	Clears cookie, 200	Pass
/api/v1/auth/verify-email	POST	Activates account, 200	Pass
/api/v1/auth/forgot-password	POST	Sends reset email, 200	Pass
/api/v1/user/profile	GET	Returns profile, 200	Pass
/api/v1/admin/stations	POST	Creates station + zone, 201	Pass
/api/v1/admin/stations	GET	Lists all stations, 200	Pass
/api/v1/admin/stations/geojson	GET	Returns GeoJSON, 200	Pass
/api/v1/admin/zones/geojson	GET	Returns GeoJSON, 200	Pass
/api/v1/admin/stations	POST (citizen role)	403 Forbidden	Pass

REFERENCES

- [1] Regina O. Obe and Leo S. Hsu. *PostGIS in Action*. Manning Publications, Shelter Island, NY, 3rd edition, 2021.
- [2] Dennis Luxen and Christian Vetter. Real-time routing with OpenStreetMap data. In *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, GIS '11, pages 513–516, New York, NY, USA, 2011. ACM.
- [3] Sisi Zlatanova, Alias Abdul Rahman, and Wenzhong Shi. Topological models and frameworks for 3D spatial objects. *Computers & Geosciences*, 30(4):419–428, 2004.
- [4] Howard Butler, Martin Daly, Alan Doyle, Sean Gillies, Stefan Hagen, and Tim Schaub. The GeoJSON format. RFC 7946, Internet Engineering Task Force (IETF), 2016. Available at <https://tools.ietf.org/html/rfc7946>.
- [5] MapLibre Contributors. MapLibre GL JS: Open-source WebGL-based vector tile maps. <https://maplibre.org>, 2023. Accessed: 2025.
- [6] Michael Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force (IETF), 2015. Available at <https://tools.ietf.org/html/rfc7519>.